

Register Allocation on SSA for AMDGPU

An optimal, decoupled allocator built on Hack's thesis

Kick-off overview — July 2026

The idea in one line

Classic graph-coloring RA (Chaitin/Briggs) treats coloring as **NP-hard** and *iterates* build → color → spill → rebuild.

On SSA form the problem becomes easy and separable:

- the interference graph is **chordal** → optimal coloring in polynomial time,
- **spilling, coloring, coalescing decouple** into a single pass — no iteration,
- register demand is known up front: it is just the **max register pressure**.

Foundation: **Hack, Grund & Goos, *Register Allocation for Programs in SSA-Form*** (CC'06) and Sebastian Hack's PhD thesis (2007).

Hack's theorem

Theorem (Hack, Grund & Goos, CC'06).

The interference graph of a program in **strict SSA form** is **chordal**.

Every chordal graph has a **perfect elimination order (PEO)**, and greedy coloring along a PEO uses the minimum number of colors
(= chromatic number = **clique number**).

Two consequences we exploit directly:

- $\chi(G) = \omega(G)$ — colors needed = size of the largest clique.
- For SSA, a PEO is *free*: it is a walk of the **dominance tree** (no MCS/LexBFS).

Why chordal? In strict SSA every value has **one definition**, so its live range is a **connected subtree** of the dominance tree — and intersection graphs of subtrees of a tree are chordal (a theorem, not a heuristic).

Illustration — live ranges on SSA

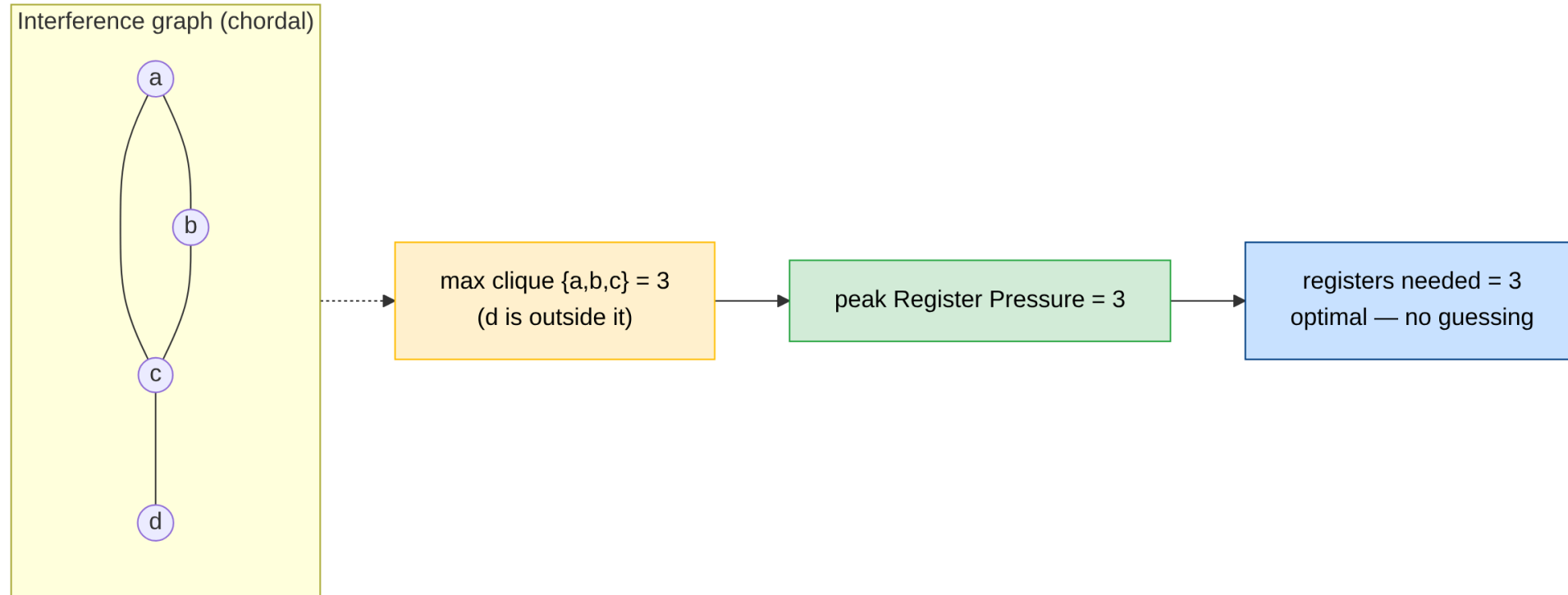
Straight-line SSA; single def $\Rightarrow$ each value's live range is **one interval** (def $\rightarrow$ last use). Two values **interfere** iff their intervals overlap.

line	program	a	b	c	d	live set	
1	a = def	█	.	.	.	{a}	
2	b = def	█	█	.	.	{a,b}	
3	c = def	█	█	█	.	{a,b,c}	← peak, RP = 3
4	.. = f(a)	█	█	█	.	{a,b,c}	(a dies)
5	.. = f(b)	.	█	█	.	{b,c}	(b dies)
6	d = def	.	.	█	█	{c,d}	
7	.. = f(c)	.	.	█	█	{c,d}	(c dies)
8	.. = f(d)	.	.	.	█	{d}	(d dies)

Overlaps $\Rightarrow$ edges **a–b**, **a–c**, **b–c**, **c–d**. At the busiest point (lines 3–4) three values are simultaneously live.

RP = max clique (demand known before we color)

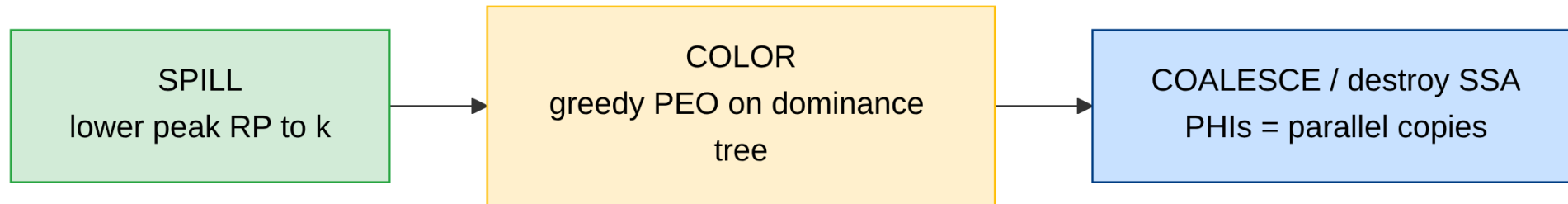
The three values live at the peak form a **clique**; **d** overlaps only **c**.



- **RP at a point** = values live there = clique size; **peak RP** = max clique.
- **Spill first** to bring peak RP to the hardware limit $k \rightarrow$ coloring is then **guaranteed** to succeed with k .

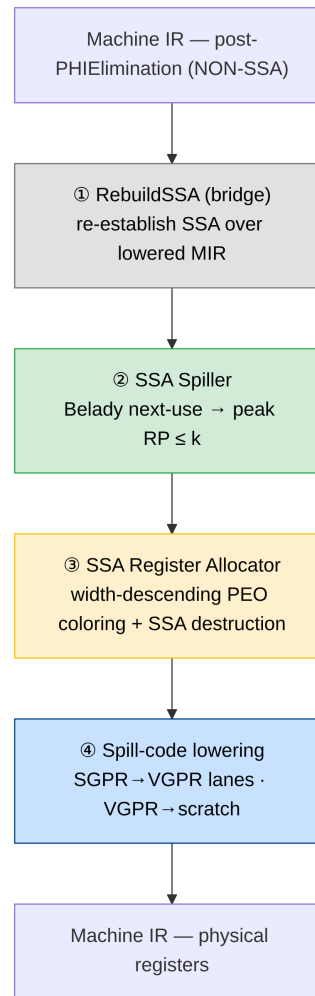
What this buys us: a decoupled, single-pass allocator

Classic allocators couple coloring and spilling and iterate. On SSA the three tasks separate cleanly and run **once**:



- **Spill** = program transformation (Belady next-use), not a graph repair.
- **Color** = one dominance-tree walk; optimal, never fails after spill.
- **Coalesce / SSA-destruction** = PHIs are register permutations. No build–color–spill–rebuild loop.

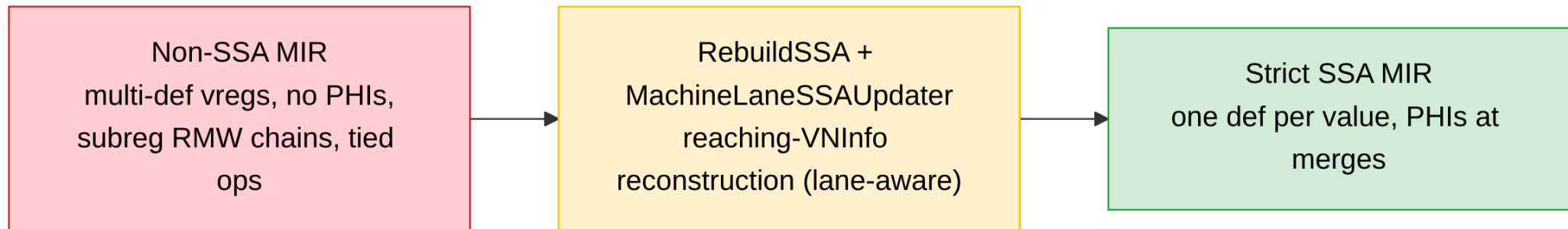
Our RA stack — four passes behind `-amdgpu-ssa-regalloc`



- SSA is preserved throughout; the spiller repairs SSA **inline** after reloads.

The critical dependency: SSA in, SSA out

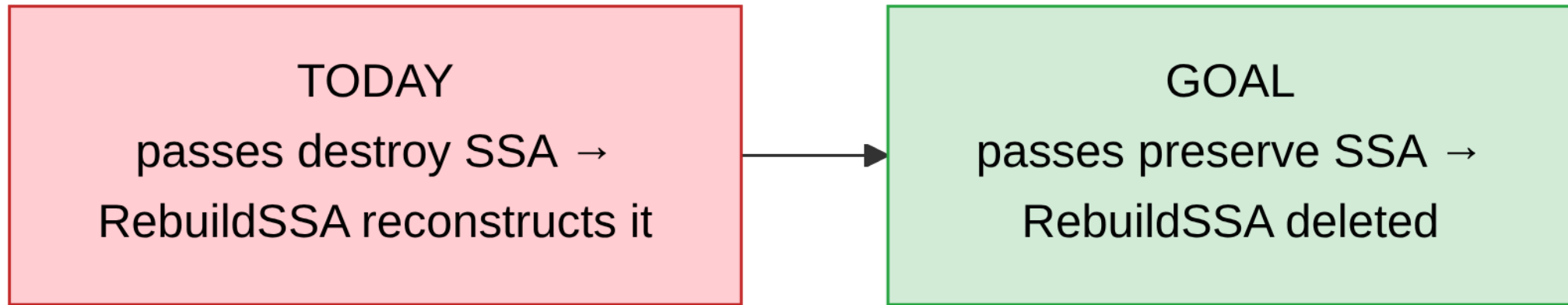
The theory needs **strict SSA input** — but LLVM's pre-RA pipeline (PHIElimination, TwoAddress, coalescing) has already **destroyed** SSA before we run. Today a temporary bridge, **RebuildSSA**, reconstructs it:



- **Why it's hard on AMDGPU:** wide tuples, subregister/lane liveness, early-clobber two-address ops (WMMA), oversized register classes with undef padding lanes.
- Reconstructing SSA from lowered MIR is **fragile and expensive** — a scaffold, not the destination. Which is exactly where the team comes in ↓

The team's mission — make SSA the native form

Goal: teach the pre-RA machine passes to **preserve SSA** instead of destroying it — so the bridge disappears and SSA is the pipeline's native form end to end.



Why it matters: the optimal-coloring guarantee *assumes* strict SSA input. Preserving it upstream removes a fragile reconstruction step, cuts compile time, and is the key to the hardest AMDGPU cases (subreg/lane liveness, WMMA, wide tuples).

Who does what: allocator / spiller / coloring core — Alex · the shared lane-aware SSA-repair engine (`MachineLaneSSAUpdater`) is common ground · **the pre-RA pass → SSA-preserving conversion is the team's charter.**

Status & what's next

Working today — full pipeline wired behind `-amdgpu-ssa-regalloc`, run end-to-end across the **3060-test** AMDGPU LIT corpus.

Area	State
Pipeline (RebuildSSA → Spiller → RA → lowering)	 live, corpus-tested
Chordal PEO coloring, SSA destruction	 implemented
Lane-aware SSA reconstruction (WMMA, padding, tied)	 hardened this cycle
Corpus crashes	103 / 3060 (from ~850)

Next:

- **PHI coalescer** (paper §4.3) — the durable fix for cross-call physreg pressure.
- **Per-class / fragmentation-aware spilling** + feasibility gate.
- Reconcile spiller vs RA register-budget accounting.

Questions?

SSARA — optimal SSA-based register allocation for AMDGPU

Design docs: [SSARA/04-Design/](#) · Foundation: Hack, Grund & Goos (CC'06) + Hack PhD (2007)